

Asymptotic Notation & Iterative Algorithm Analysis

Learn to answer one big question:

“How fast does an algorithm grow when input becomes huge?”

CS332 – Design and Analysis of Algorithms

Dr. Zakir Khan

Fall 2026

Week 04 at a Glance

Session 10

Big-O

Upper bound
Ignore small details
Focus on growth

Session 11

Ω , Θ , o , ω

Lower bound
Tight bound
Strict growth ideas

Session 12

Iterative Analysis

Single loops
Nested loops
Summations

Remember

By the end of Week 04, you should be able to look at a simple algorithm and explain its growth using asymptotic notation.

Learning Targets – In Simple Words

- ① I can explain what **Big-O** means.
- ② I can tell the difference between **upper**, **lower**, and **tight** bounds.
- ③ I can simplify running-time expressions such as

$$4n^2 + 7n + 20 \Rightarrow \Theta(n^2).$$

- ④ I can count how many times loops execute.
- ⑤ I can analyze **single**, **consecutive**, and **nested** loops.
- ⑥ I can use simple summations such as

$$1 + 2 + \cdots + n = \frac{n(n+1)}{2}.$$

Simple Idea

Think of this week as learning the “speed language” of algorithms.

Why Do We Need Big-O?

Simple Idea

Suppose two programs solve the same problem.

- Program A: finishes in 1 second for 100 items.
- Program B: finishes in 2 seconds for 100 items.

Which is better for **1,000,000 items**?

Small input

↓ Growth matters!

Huge input

Big-O helps us compare
how running time grows.

Remember

Big-O is mainly about **growth** as n becomes **very large**, not about one small test.

A Very Simple Analogy

Imagine a School Bus

A bus can carry at most 50 students.
If we say:

$$\text{Students} \leq 50$$

then 50 is an **upper bound**.

Big-O is Similar

If the running time is eventually no worse than

$$c \cdot g(n),$$

then we write:

$$T(n) = O(g(n)).$$

Remember

Big-O = asymptotic upper bound

It tells us a growth rate the algorithm will not exceed after some point.

Big-O – Friendly Definition

$$T(n) = O(g(n))$$

means:

“For sufficiently large n , $T(n)$ grows **no faster than a constant multiple** of $g(n)$.”

Math Step

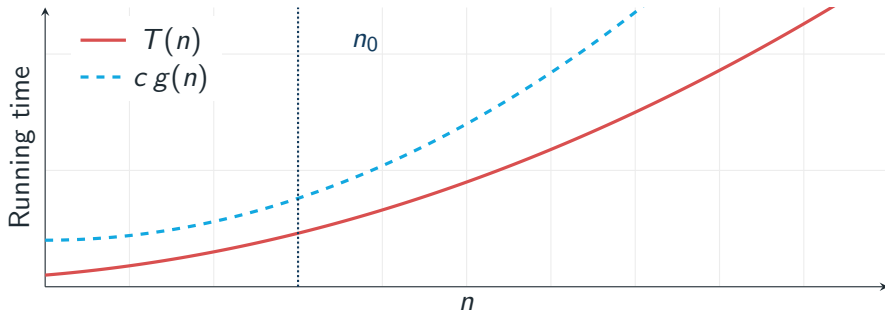
There exist constants $c > 0$ and n_0 such that

$$0 \leq T(n) \leq c g(n) \quad \text{for all } n \geq n_0.$$

Simple Idea

c lets us stretch the comparison curve. n_0 says: “Start caring from this point onward.”

Big-O Picture: An Upper Ceiling



Remember

After n_0 , the blue curve stays above the red curve. That is the idea of an **upper bound**.

The Golden Rule: Keep the Fastest-Growing Term

$$T(n) = 4n^2 + 7n + 20$$

Math Step

Step 1: The terms are $4n^2$, $7n$, and 20.

Step 2: For huge n , the n^2 term grows fastest.

Step 3: Ignore the constant multiplier: $4n^2 \Rightarrow n^2$.

Result:

$$T(n) = O(n^2)$$

Remember

Drop constant factors and lower-order terms. Keep the dominant growth term.

Why Can We Ignore Small Terms?

n	n^2	$10n$	1000
10	100	100	1000
100	10,000	1,000	1000
1000	1,000,000	10,000	1000
10,000	100,000,000	100,000	1000

Simple Idea

As n gets huge:

$$n^2 \gg n \gg 1$$

So in

$$4n^2 + 7n + 20,$$

the n^2 term controls the growth.

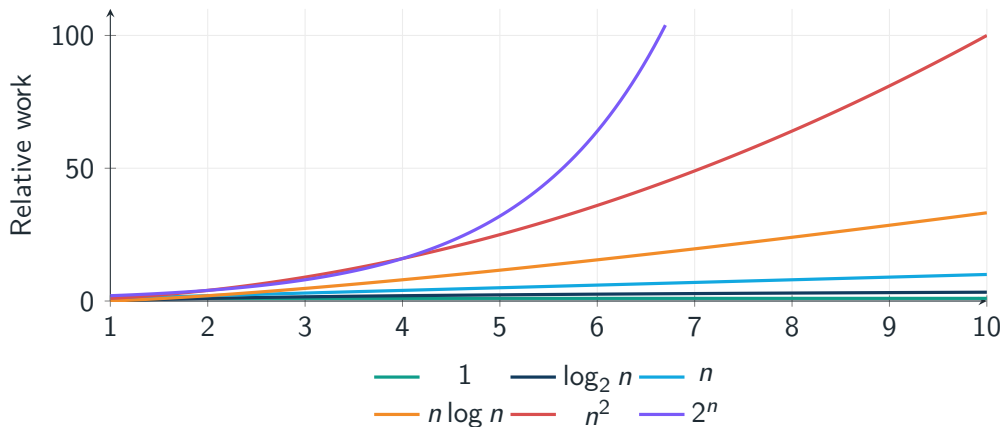
Common Growth Rates

Complexity	Name	Simple Meaning
$O(1)$	Constant	Same work
$O(\log n)$	Logarithmic	Cut problem again and again
$O(n)$	Linear	One pass
$O(n \log n)$	Linearithmic	Efficient sorting style
$O(n^2)$	Quadratic	Compare many pairs
$O(n^3)$	Cubic	Three nested dimensions
$O(2^n)$	Exponential	Doubles with each extra item
$O(n!)$	Factorial	Tries almost every ordering

Common Mistake

A smaller growth class usually scales better for large inputs.

Growth Curves – Which One Explodes?



Example 1: One Simple Loop

```
1 for (int i = 0; i < n; i++) {  
2     cout << i;  
3 }
```

Math Step

The loop body runs:

n times

If one iteration costs a constant c , then:

$$T(n) = cn + d$$

Drop constants:

$$T(n) = O(n)$$

Simple Idea

One full pass through n items $\Rightarrow$ usually linear time.

Example 2: Two Nested Loops

```
1 for (int i = 0; i < n; i++) {  
2     for (int j = 0; j < n; j++) {  
3         cout << i << j;  
4     }  
5 }
```

Math Step

Outer loop: n times

Inner loop: n times for each outer iteration

Total:

$$n \times n = n^2$$

Therefore:

$$T(n) = O(n^2)$$

Big-O Checklist

When you see code, ask:

- ① What is the **input size** n ?
- ② What operation repeats the most?
- ③ How many times does it repeat?
- ④ Write a simple running-time expression.
- ⑤ Keep the fastest-growing term.
- ⑥ Remove constant multipliers.

Remember

$$6n^3 + 20n^2 + 5n + 100 \Rightarrow O(n^3)$$

Five Symbols – One Family

Symbol	Meaning	Memory Word
$O(g(n))$	Upper bound	Ceiling
$\Omega(g(n))$	Lower bound	Floor
$\Theta(g(n))$	Tight bound	Exact growth class
$o(g(n))$	Strictly smaller growth	Definitely slower
$\omega(g(n))$	Strictly larger growth	Definitely faster

Simple Idea

Think: **ceiling, floor, exact fit, strictly below, strictly above.**

Omega Ω : The Floor

Simple Meaning

$$T(n) = \Omega(g(n))$$

means:

For large n , $T(n)$ grows **at least as fast as** a constant multiple of $g(n)$.

Math Step

There exist $c > 0$ and n_0 such that:

$$0 \leq c g(n) \leq T(n)$$

for all $n \geq n_0$.

Remember

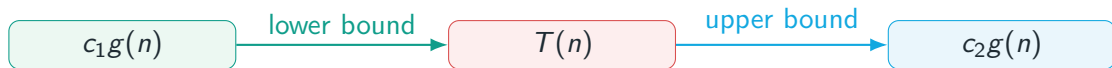
Ω = asymptotic lower bound

Theta Θ : The Exact Growth Class

$$T(n) = \Theta(g(n))$$

means $T(n)$ is trapped between two constant multiples of $g(n)$:

$$c_1g(n) \leq T(n) \leq c_2g(n) \quad (n \geq n_0)$$

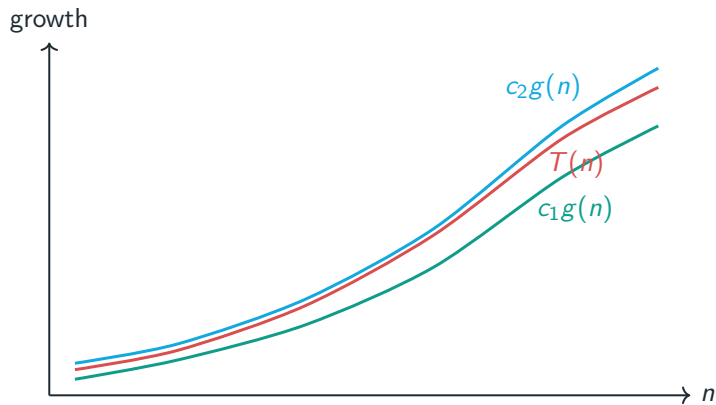


Remember

If both $O(g(n))$ and $\Omega(g(n))$ are true, then:

$$T(n) = \Theta(g(n))$$

Big-O vs Omega vs Theta



Simple Idea

If $T(n)$ stays between a scaled lower and upper copy of $g(n)$, then $T(n) = \Theta(g(n))$.

Example: Find O , Ω , and Θ

Given:

$$T(n) = 3n^2 + 10n + 7$$

Upper

$$T(n) = O(n^2)$$

Lower

$$T(n) = \Omega(n^2)$$

Tight

$$T(n) = \Theta(n^2)$$

Math Step

The dominant term is n^2 , so the true asymptotic growth class is quadratic.

Little- o : Strictly Smaller Growth

$$f(n) = o(g(n))$$

means $f(n)$ grows **strictly slower** than $g(n)$.

A useful limit test:

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0$$

True

$$n = o(n^2)$$

because

$$\frac{n}{n^2} = \frac{1}{n} \rightarrow 0.$$

Not True

$$n^2 = o(n^2)$$

because

$$\frac{n^2}{n^2} = 1 \neq 0.$$

Little- ω : Strictly Larger Growth

$$f(n) = \omega(g(n))$$

means $f(n)$ grows **strictly faster** than $g(n)$.

Useful limit test:

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = \infty$$

True

$$n^2 = \omega(n)$$

because

$$\frac{n^2}{n} = n \rightarrow \infty.$$

Not True

$$n = \omega(n)$$

because

$$\frac{n}{n} = 1.$$

One Comparison Table to Remember

Notation	Question	Quick Memory
$O(g(n))$	Can $g(n)$ be an upper ceiling?	no faster than
$\Omega(g(n))$	Can $g(n)$ be a lower floor?	no slower than
$\Theta(g(n))$	Same growth class?	tight / exact class
$o(g(n))$	Definitely slower?	strict upper
$\omega(g(n))$	Definitely faster?	strict lower

Remember

$$\Theta(g(n)) \subseteq O(g(n)) \quad \text{and} \quad \Theta(g(n)) \subseteq \Omega(g(n)).$$

Proof-Style Example for Big-O

Show:

$$3n + 2 = O(n)$$

We need:

$$3n + 2 \leq cn$$

For $n \geq 1$:

$$2 \leq 2n$$

So:

$$3n + 2 \leq 3n + 2n = 5n$$

Choose:

$$c = 5, \quad n_0 = 1$$

Therefore:

$$\boxed{3n + 2 = O(n)}$$

Proof-Style Example for Theta

Show:

$$T(n) = 4n^2 + 3n + 7 = \Theta(n^2)$$

Math Step

For $n \geq 1$:

$$4n^2 \leq T(n)$$

and because $3n \leq 3n^2$ and $7 \leq 7n^2$,

$$T(n) \leq 4n^2 + 3n^2 + 7n^2 = 14n^2.$$

Therefore:

$$4n^2 \leq T(n) \leq 14n^2.$$

Choose $c_1 = 4$, $c_2 = 14$, $n_0 = 1$; hence

$$T(n) = \Theta(n^2).$$

Quick Classification Practice

Classify each statement as **True** or **False**.

- ① $n = O(n^2)$
- ② $n^2 = O(n)$
- ③ $5n^2 + 3 = \Theta(n^2)$
- ④ $\log n = o(n)$
- ⑤ $n^3 = \omega(n^2)$

Try It Yourself

Do not guess from symbols. Compare the growth rates.

Practice Answers

① $n = O(n^2)$

True

② $n^2 = O(n)$

False

③ $5n^2 + 3 = \Theta(n^2)$

True

④ $\log n = o(n)$

True

⑤ $n^3 = \omega(n^2)$

True

Remember

Growth hierarchy:

$$1 \prec \log n \prec n \prec n \log n \prec n^2 \prec n^3 \prec 2^n \prec n!$$

How to Analyze a Loop

Use this **5-step recipe**:

- ① Find input size n .
- ② Identify the repeated operation.
- ③ Count how many times it runs.
- ④ Write the total work as a formula or sum.
- ⑤ Simplify to $\Theta(\cdot)$ or $O(\cdot)$.

Simple Idea

For loops, the main question is:

How many total iterations happen?

Single Loop: Linear Time

```
1 for (int i = 0; i < n; i++) {  
2     x = x + 1;  
3 }
```

Math Step

The body runs once for:

$$i = 0, 1, 2, \dots, n - 1$$

Number of iterations:

$$n$$

So:

$$T(n) = an + b$$

Therefore:

$$T(n) = \Theta(n)$$

Loop with Step Size 2

```
1 for (int i = 0; i < n; i += 2) {  
2     x++;  
3 }
```

Values of i :

0, 2, 4, 6, ...

Approximate number of iterations:

$$\frac{n}{2}$$

So:

$$T(n) = c \frac{n}{2} + d$$

Drop constants:

$$T(n) = \Theta(n)$$

Common Mistake

$\frac{n}{2}$ is still linear. Constants do not change the asymptotic class.

Consecutive Loops: Add the Work

```
1 for (int i = 0; i < n; i++) A[i] = 0;  
2 for (int j = 0; j < n; j++) B[j] = 1;
```

Math Step

First loop does n iterations. Second loop does n iterations.

$$T(n) = n + n = 2n$$

Drop the constant 2:

$$T(n) = \Theta(n)$$

Simple Idea

Consecutive loops are usually added, not multiplied.

Nested Loops: Multiply the Work

```
1 for (int i = 0; i < n; i++) {  
2     for (int j = 0; j < n; j++) {  
3         count++;  
4     }  
5 }
```

$$\underbrace{n}_{\text{outer}} \times \underbrace{n}_{\text{inner}} = n^2$$

$$T(n) = \Theta(n^2)$$

Simple Idea

If the inner loop always runs n times for each of n outer iterations, multiply.

Triangular Nested Loop

```
1 for (int i = 1; i <= n; i++) {  
2     for (int j = 1; j <= i; j++) {  
3         count++;  
4     }  
5 }
```

Inner loop runs:

$$1, 2, 3, \dots, n$$

Total:

$$T(n) = 1 + 2 + 3 + \dots + n$$

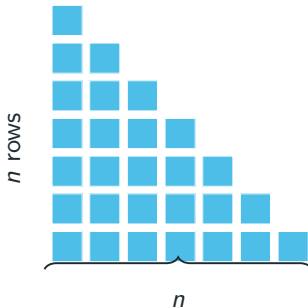
Use:

$$\sum_{i=1}^n i = \frac{n(n+1)}{2}$$

Therefore:

$$T(n) = \frac{n^2 + n}{2} \Rightarrow \boxed{\Theta(n^2)}$$

Why $1 + 2 + \dots + n$ Becomes Quadratic



$$1 + 2 + \dots + n = \frac{n(n+1)}{2} \approx \frac{n^2}{2}$$

Remember

A triangular amount of work still grows like n^2 .

Logarithmic Loop: Keep Doubling

```
1 for (int i = 1; i < n; i = i * 2) {  
2     count++;  
3 }
```

Values:

1, 2, 4, 8, 16, ...

After k iterations:

$$2^k \approx n$$

Take $\log_2$:

$$k \approx \log_2 n$$

Therefore:

$$T(n) = \Theta(\log n)$$

Simple Idea

Repeated doubling or halving usually means logarithmic time.

Nested n and $\log n$ Loops

```
1 for (int i = 0; i < n; i++) {  
2     for (int j = 1; j < n; j *= 2) {  
3         count++;  
4     }  
5 }
```

Outer loop:

n

Inner loop:

$\log_2 n$

Total:

$n \log_2 n$

Therefore:

$$T(n) = \Theta(n \log n)$$

Three Nested Loops

```
1 for (int i = 0; i < n; i++)  
2     for (int j = 0; j < n; j++)  
3         for (int k = 0; k < n; k++)  
4             count++;
```

$$n \times n \times n = n^3$$

$$T(n) = \Theta(n^3)$$

Remember

Three independent loops of size n usually give cubic growth.

A Summation View

Sometimes the inner-loop work changes with i . Then use a sum:

$$T(n) = \sum_{i=1}^n f(i)$$

Sum	Growth
$\sum_{i=1}^n 1 = n$	$\Theta(n)$
$\sum_{i=1}^n i = \frac{n(n+1)}{2}$	$\Theta(n^2)$
$\sum_{i=1}^n i^2 = \frac{n(n+1)(2n+1)}{6}$	$\Theta(n^3)$

Simple Idea

A summation is simply a careful way to count the total number of repeated operations.

Best and Worst Case: Linear Search

Search for key in an array:

```
1 for (int i = 0; i < n; i++) {  
2     if (A[i] == key)  
3         return i;  
4 }  
5 return -1;
```

Best Case

Found at first item:

$$\Theta(1)$$

Worst Case

Found last or absent:

$$\Theta(n)$$

Typical Idea

Several checks:

$$\Theta(n)$$

Best Case vs Worst Case – Picture



Best case = $\Theta(1)$, Worst case = $\Theta(n)$

Remember

The same algorithm can have different running times on different inputs of the same size.

Consecutive vs Nested – Do Not Mix Them Up

Consecutive

Loop 1: n

Loop 2: n^2

Total:

$$n + n^2$$

Dominant term:

$$\Theta(n^2)$$

Nested

Outer: n

Inner: n

Total:

$$n \times n$$

Therefore:

$$\Theta(n^2)$$

Common Mistake

Consecutive loops: usually **add**. Nested loops: usually **multiply** or use a summation.

Step-by-Step Worked Example

Suppose:

$$T(n) = 5 + 3n + 2n^2$$

① Identify terms:

$$5, 3n, 2n^2$$

② Dominant term:

$$2n^2$$

③ Remove constant multiplier:

$$n^2$$

④ Final tight bound:

$$T(n) = \Theta(n^2)$$

Remember

When you know the exact dominant growth, prefer Θ .

Challenge 1

Analyze:

```
1 for (int i = 0; i < n; i++) {  
2     for (int j = 0; j < 10; j++) {  
3         count++;  
4     }  
5 }
```

Try It Yourself

Outer loop: _____

Inner loop: _____

Total: _____

Complexity: _____

Challenge 1 – Answer

Outer loop:

$$n$$

Inner loop:

$$10$$

Total:

$$10n$$

Drop the constant:

$$\Theta(n)$$

Simple Idea

A constant-size inner loop does **not** create n^2 .

Challenge 2

Analyze:

```
1 for (int i = n; i > 1; i /= 2) {  
2     count++;  
3 }
```

Try It Yourself

Think: n , $n/2$, $n/4$, $n/8$, ...

How many halvings until we reach 1?

Challenge 2

Analyze:

```
1 for (int i = n; i > 1; i /= 2) {  
2     count++;  
3 }
```

Try It Yourself

Think: n , $n/2$, $n/4$, $n/8$, ...

How many halvings until we reach 1?

$$\Theta(\log n)$$

Challenge 3: Dependent Nested Loop

```
1 for (int i = 1; i <= n; i++) {  
2     for (int j = 1; j <= i*i; j++) {  
3         count++;  
4     }  
5 }
```

Simple Idea

For each value of i , the inner loop runs i^2 times.

Math Step

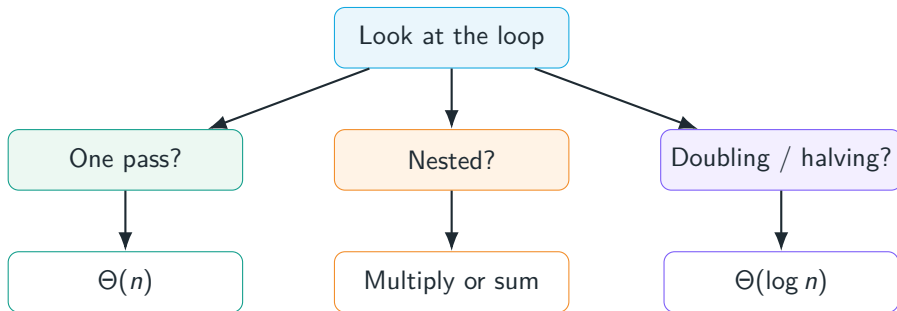
Total work:

$$\sum_{i=1}^n i^2 = \frac{n(n+1)(2n+1)}{6}$$

Dominant term is proportional to n^3 .
Therefore:

$$\Theta(n^3)$$

A Fast Decision Tree



Remember

If the inner-loop limit depends on the outer-loop variable, use a summation.

Mini Worksheet – 5 Minutes

Find the tight asymptotic complexity.

- ① $7n + 100$
- ② $3n^2 + 4n + 1$
- ③ Two consecutive loops: n and n^2
- ④ Nested loops: outer n , inner $n/2$
- ⑤ Loop variable doubles each time
- ⑥ $1 + 2 + \dots + n$

Try It Yourself

Write only the final class first. Then explain why in one sentence.

Mini Worksheet – Answers

- ① $7n + 100 \Rightarrow \Theta(n)$
- ② $3n^2 + 4n + 1 \Rightarrow \Theta(n^2)$
- ③ $n + n^2 \Rightarrow \Theta(n^2)$
- ④ $n \cdot (n/2) \Rightarrow \Theta(n^2)$
- ⑤ Doubling loop $\Rightarrow \Theta(\log n)$
- ⑥ $1 + 2 + \dots + n \Rightarrow \Theta(n^2)$

Common Mistake

Do not answer only from the number of loops. Always inspect loop limits and update rules.

Quiz 1 Preparation – What You Must Be Able to Do

- Explain Big-O, Ω , and Θ in words.
- Distinguish tight vs non-tight bounds.
- Simplify polynomial running-time expressions.
- Compare growth classes.
- Analyze single and nested loops.
- Use summation for dependent loops.
- Recognize best-case and worst-case behavior.

Remember

Do not memorize only symbols. Practice **explaining why** a complexity is correct.

Week 04 Final Summary

Bounds

O = ceiling

Ω = floor

Θ = tight

Strict Growth

o = strictly slower

ω = strictly faster

Loops

Add consecutive

Multiply nested

Sum dependent

Count work $\rightarrow$ form $T(n)$ $\rightarrow$ keep dominant growth

Exit Ticket

Before leaving, answer these three questions:

- ① Why is $5n^2 + 100n + 7$ in $\Theta(n^2)$?
- ② What is the difference between $O(n^2)$ and $\Theta(n^2)$?
- ③ What is the complexity of:

$$\sum_{i=1}^n i$$

One-Sentence Goal

If you can explain these without memorized definitions, you understand Week 04.

End of Week 04

Count. Compare. Simplify. Explain.

Next: Recursion and Recurrence Relations